

Listas em Python — Exercícios de Engenharia

append · remove · copy · extend

Engenharia Mecânica · Elétrica · Telecomunicações

Nome: Lucas Viana de Freitas

Data: 24 / 04 / 2026

Turma / Curso: Engenharia Elétrica

Nota: _____

Referência rápida — métodos de lista:

Método	Sintaxe	O que faz
<code>lista.append(x)</code>	<code>lista.append(42.5)</code>	Adiciona x ao final da lista
<code>lista.remove(x)</code>	<code>lista.remove(42.5)</code>	Remove a 1ª ocorrência de x
<code>lista.pop(i)</code>	<code>lista.pop(0)</code>	Remove e retorna o elemento no índice i
<code>lista.pop()</code>	<code>lista.pop()</code>	Remove e retorna o último elemento
<code>lista.extend(L)</code>	<code>lista.extend([1.0, 2.0])</code>	Acrescenta todos os itens de L ao final
<code>lista.copy()</code>	<code>b = lista.copy()</code>	Cria cópia independente da lista
<code>lista[:]</code>	<code>b = lista[:]</code>	Cópia via fatiamento — mesmo efeito
<code>lista.insert(i,x)</code>	<code>lista.insert(0, 99.9)</code>	Insere x na posição i
<code>lista.index(x)</code>	<code>lista.index(42.5)</code>	Retorna índice da 1ª ocorrência de x
<code>len(lista)</code>	<code>len(lista)</code>	Retorna o número de elementos

Bloco 1 — `append()` Adicionar elementos à lista

Ex. 1	<code>append</code>	Eng. Mecânica — Registro de temperatura de forno industrial	básico
----------	---------------------	---	--------

Um forno industrial é monitorado a cada hora durante um turno de 6 horas. Escreva um programa que:

- Crie uma lista vazia chamada temperaturas
- Use `append()` para inserir as 6 leituras fornecidas abaixo
- Exiba a lista completa, o valor máximo e o valor mínimo

Dados de entrada

leituras (°C): 820, 835, 848, 791, 812, 856

Saída esperada:

» Leituras: [820, 835, 848, 791, 812, 856]

Máximo:	856 °C
Mínimo:	791 °C

Código Python:

```
temperaturas = []
leituras = [820, 835, 848, 791, 812, 856]

for t in leituras:
    temperaturas.append(t)

print("Leituras:", temperaturas)
print("Máximo:", max(temperaturas), "°C")
print("Mínimo:", min(temperaturas), "°C")
```

Ex.
2

append

Eng. Elétrica — Coleta de amostras de tensão em barramento

básico

Um osciloscópio registra amostras de tensão em um barramento de 12 V. Use `append()` dentro de um laço `for` para acumular as leituras em uma lista. Ao final, calcule a média e exiba alerta para qualquer leitura fora da faixa $\pm 10\%$ (entre 10,8 V e 13,2 V).

Dados de entrada

amostras (V): 11.9, 12.1, 12.0, 13.5, 11.7, 12.3, 10.5, 12.0

Saída esperada:

```
>> Média: 12.01 V
    ALERTA: 13.5 V fora da faixa!
    ALERTA: 10.5 V fora da faixa!
```



Dica: Itere sobre a lista de amostras com `for` e use `append()` a cada iteração para construir a lista de leituras.

Código Python:

```
amostras = [11.9, 12.1, 12.0, 13.5, 11.7, 12.3, 10.5, 12.0]
leituras = []

for v in amostras:
```

```
leituras.append(v)
```

```
media = sum(leituras) / len(leituras)
```

```
print("Média:", round(media, 2), "V")
```

```
for v in leituras:
```

```
    if v < 10.8 or v > 13.2:
```

```
        print("ALERTA:", v, "V fora da faixa!")
```

Ex.
3

append

Eng. Telecomunicações — Buffer de pacotes
recebidos

intermediário

Um roteador recebe pacotes com tamanhos variados (em bytes). Use `append()` dentro de um laço para armazená-los em um buffer e calcule:

- Total de pacotes recebidos
- Tamanho total em bytes
- Porcentagem de pacotes acima de 1500 bytes (pacotes jumbo)

Dados de entrada

tamanhos (bytes): 512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024

Saída esperada:

```
>> Pacotes recebidos : 10
    Total de bytes   : 10946
    Pacotes jumbo    : 1 (10.0%)
```



Dica: Conte os pacotes jumbo com um acumulador ou com `sum()` e uma condição dentro do gerador.

Código Python:

```
tamanhos = [512, 1024, 1500, 1518, 256, 1500, 2048, 64, 1500, 1024]
```

```
buffer = []
```

```
for t in tamanhos:
```

```
    buffer.append(t)
```

```
total = len(buffer)
```

```
bytes_total = sum(buffer)
```

```
jumbo = sum(1 for t in buffer if t > 1500)
```

```
print("Pacotes:", total)
```

```
print("Bytes:", bytes_total)
```

```
print("Jumbo:", jumbo, f'({jumbo/total*100:.1f}%)')
```

Bloco 2 — remove() · pop() Remover elementos da lista

Ex.
4

remove

Eng. Mecânica — Descarte de leituras com defeito de sensor

básico

Um acelerômetro gera o valor sentinela `-999.0` quando há falha de comunicação. Escreva um programa que remova todos os valores `-999.0` da lista usando `remove()` em laço `while` e exiba a lista limpa com a quantidade de leituras válidas.

Dados de entrada

vibracao (m/s²): 0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67

Saída esperada:

```
>> Leituras válidas: [0.12, 0.34, 0.28, 0.91, 1.42, 0.67]
    Quantidade      : 6
```



Dica: `remove()` elimina apenas a primeira ocorrência — use `while (-999.0 in lista)` para eliminar todas.

Código Python:

```
vibracao = [0.12, 0.34, -999.0, 0.28, 0.91, -999.0, 1.42, 0.67]
```

```
while -999.0 in vibracao:
```

```
    vibracao.remove(-999.0)
```

```
print(vibracao)
```

```
print("Quantidade:", len(vibracao))
```

**Ex.
5****remove****Eng. Elétrica — Gerenciamento de cargas no sistema elétrico****intermediário**

Um sistema de gerenciamento de energia mantém uma lista de cargas ativas em kW. Quando uma carga é desligada, ela é removida. Escreva um programa que:

- Remova a carga de 8.0 kW (desligada por sobrecarga)
- Remova a carga de 40.0 kW (equipamento em manutenção)
- Exiba a potência total antes e depois, e a redução obtida

Dados de entrada

cargas (kW): 15.0, 22.5, 8.0, 40.0, 12.5, 30.0

Saída esperada:

```
>> Cargas ativas : [15.0, 22.5, 12.5, 30.0]
    Potência antes: 128.0 kW
    Potência agora: 80.0 kW
    Redução       : 48.0 kW
```

Código Python:

```
cargas = [15.0, 22.5, 8.0, 40.0, 12.5, 30.0]
```

```
antes = sum(cargas)
```

```
cargas.remove(8.0)
```

```
cargas.remove(40.0)
```

```
depois = sum(cargas)
```

```
print(cargas)
```

```
print("Antes:", antes)
```

```
print("Agora:", depois)
```

```
print("Redução:", antes - depois)
```

**Ex.
6****remove****Eng. Telecomunicações — Desativação de canais com interferência****intermediário**

Uma estação rádio-base opera em múltiplos canais (MHz). Canais com nível de interferência acima de -70 dBm devem ser desativados. Construa a lista de canais ativos, identifique os interferidos e remova-os, exibindo cada canal desativado e os canais que restaram.

Dados de entrada

```
canais (MHz)      : 850, 900, 950, 1800, 2100, 2600
interferência (dBm): 850→-65, 900→-80, 950→-72, 1800→-55, 2100→-90, 2600→-68
limiar de corte   : -70 dBm (acima desse valor → desativar)
```

Saída esperada:

```
>> Canal 850 MHz desativado (interferência: -65 dBm)
     Canal 950 MHz desativado (interferência: -72 dBm)
     Canal 1800 MHz desativado (interferência: -55 dBm)
     Canal 2600 MHz desativado (interferência: -68 dBm)
     Canais ativos: [900, 2100]
```



Dica: Nunca remova elementos de uma lista enquanto itera sobre ela com for — monte uma lista auxiliar dos itens a remover e depois chame `remove()` fora do laço.

Código Python:

```
canais = [850, 900, 950, 1800, 2100, 2600]
interf = {850:-65, 900:-80, 950:-72, 1800:-55, 2100:-90, 2600:-68}

remover = []

for c in canais:
    if interf[c] > -70:
        print("Canal", c, "desativado")
        remover.append(c)

for c in remover:
    canais.remove(c)

print("Ativos:", canais)
```

Bloco 3 — copy() Copiar listas com independência

Ex.
7

copy

Eng. Mecânica — Backup de parâmetros de
usinagem antes de ajuste

básico

Antes de ajustar os parâmetros de velocidade de corte (m/min) de uma máquina CNC, o operador deve preservar os valores originais. Escreva um programa que:

- Crie um backup com `copy()` antes de qualquer modificação
- Altere o primeiro valor da lista original para 130.0
- Exiba ambas as listas e explique (em comentário no código) por que o backup não foi alterado
- Mostre também o que aconteceria se tivesse usado `=` em vez de `copy()`

Dados de entrada

```
params_originais (m/min): 120.0, 150.0, 95.0, 200.0, 110.0
novo valor para índice 0 : 130.0
```

Saída esperada:

```
>> Original (modificada): [130.0, 150.0, 95.0, 200.0, 110.0]
Backup (preservado): [120.0, 150.0, 95.0, 200.0, 110.0]
```



Dica: `lista_b = lista_a` não cria uma cópia — cria um segundo nome para o mesmo objeto em memória. Qualquer alteração aparece nos dois nomes.

Código Python:

```
original = [120.0, 150.0, 95.0, 200.0, 110.0]
```

```
backup = original.copy()
```

```
original[0] = 130.0
```

```
print("Original:", original)
```

```
print("Backup:", backup)
```

Ex.
8

copy

Eng. Elétrica — Simulação de curto-circuito sem
alterar o circuito nominal

intermediário

Um engenheiro quer simular a falha de um resistor (curto-circuito: $R \rightarrow 0 \Omega$) sem alterar a configuração nominal. Escreva um programa que:

- Copie a lista nominal antes da simulação
- Na cópia, substitua o resistor de 100.0 Ω por 0.0 Ω
- Calcule a resistência equivalente em série (soma) nos dois casos
- Exiba nominal, simulação e a diferença entre elas

Dados de entrada

```
nominal ( $\Omega$ ): 47.0, 100.0, 220.0, 33.0, 68.0
resistor em curto: 100.0  $\Omega \rightarrow 0.0 \Omega$ 
```

Saída esperada:

»	Nominal : [47.0, 100.0, 220.0, 33.0, 68.0] Simulação : [47.0, 0.0, 220.0, 33.0, 68.0] Req nominal : 468.0 Ω Req falha : 368.0 Ω Variação : 100.0 Ω
	Dica: Use <code>lista.index(valor)</code> para localizar a posição do resistor de 100.0 Ω e então use indexação para substituí-lo.

Código Python:

```
nominal = [47.0, 100.0, 220.0, 33.0, 68.0]
sim = nominal.copy()

i = sim.index(100.0)
sim[i] = 0.0

print("Nominal:", nominal)
print("Simulação:", sim)
print("Req nominal:", sum(nominal))
print("Req falha:", sum(sim))
```

Ex. 9	copy	Eng. Telecomunicações — Comparação de tabelas de roteamento	avançado
----------	------	---	----------

Após uma atualização de protocolo, a tabela de rotas de um roteador muda. Escreva um programa que:

- Copie a tabela antiga antes de qualquer modificação
- Aplique as mudanças na cópia (remova a rota obsoleta, adicione as duas novas)
- Compare as duas versões e liste rotas adicionadas e rotas removidas

Dados de entrada	
tabela atual	: '10.0.0.0/8', '172.16.0.0/12', '192.168.1.0/24', '203.0.113.0/24', '198.51.100.0/24'
rota removida	: '203.0.113.0/24'
rotas inseridas:	'10.10.0.0/16' e '192.168.100.0/24'

Saída esperada:

»	Rotas adicionadas: ['10.10.0.0/16', '192.168.100.0/24'] Rotas removidas : ['203.0.113.0/24']
---	---



Dica: Para encontrar diferenças entre duas listas, use list comprehension com in: [r for r in nova if r not in antiga].

Código Python:

```
antiga = ['10.0.0.0/8','172.16.0.0/12','192.168.1.0/24','203.0.113.0/24','198.51.100.0/24']
```

```
nova = antiga.copy()
```

```
nova.remove('203.0.113.0/24')
```

```
nova.append('10.10.0.0/16')
```

```
nova.append('192.168.100.0/24')
```

```
add = [r for r in nova if r not in antiga]
```

```
rem = [r for r in antiga if r not in nova]
```

```
print("Adicionadas:", add)
```

```
print("Removidas:", rem)
```

Bloco 4 — extend() Concatenar listas

Ex.
10

extend

Eng. Mecânica — Consolidação de leituras de múltiplos sensores de pressão

básico

Três sensores de pressão monitoram diferentes pontos de uma tubulação. As leituras estão em listas separadas. Use extend() para consolidar tudo em uma única lista e calcule média, máximo e mínimo do sistema.

Dados de entrada

```
sensor_A (bar): 3.2, 3.4, 3.1, 3.5  
sensor_B (bar): 4.1, 4.0, 4.3, 3.9  
sensor_C (bar): 2.8, 2.9, 3.0, 2.7
```

Saída esperada:

»

```
Total de leituras: 12  
Pressão média      : 3.41 bar  
Pressão máxima     : 4.3 bar  
Pressão mínima     : 2.7 bar
```



Dica: extend() é diferente de append(): append(lista) insere a lista como um único elemento, criando lista aninhada. extend(lista) insere cada elemento individualmente.

Código Python:

```
A = [3.2,3.4,3.1,3.5]
B = [4.1,4.0,4.3,3.9]
C = [2.8,2.9,3.0,2.7]

total = A.copy()
total.extend(B)
total.extend(C)

print(len(total))
print("Média:", sum(total)/len(total))
print("Máx:", max(total))
print("Min:", min(total))
```

Ex. 11	extend	Eng. Elétrica — Fusão e análise de medições de dois multímetros	intermediário
-----------	--------	---	---------------

Dois técnicos mediram a resistência de uma bobina com multímetros diferentes. Una as medições com `extend()`, ordene a lista resultante, calcule a média e identifique valores que desviam mais de $\pm 15\%$ da média.

Dados de entrada
medidor_1 (Ω): 48.2, 51.0, 49.7, 47.5, 52.3 medidor_2 (Ω): 50.1, 46.8, 53.0, 49.0, 55.2

Saída esperada:

»	Medições ordenadas: [46.8, 47.5, 48.2, 49.0, 49.7, 50.1, 51.0, 52.3, 53.0, 55.2] Média: 50.28 Ω Tolerância: $\pm 7.54 \Omega$ (nenhum valor fora da faixa neste conjunto)
	Dica: Use <code>medidor_1.copy()</code> antes de <code>extend()</code> para não modificar a lista original do medidor 1.

Código Python:

```
m1 = [48.2,51.0,49.7,47.5,52.3]
m2 = [50.1,46.8,53.0,49.0,55.2]
```

```

dados = m1.copy()
dados.extend(m2)
dados.sort()

media = sum(dados)/len(dados)
tol = media * 0.15

print(dados)
print("Média:", round(media,2))

```

**Ex.
12**

extend

**Eng. Telecomunicações — Agregação de logs de
potência de antenas 5G**

avanzado

Três antenas 5G registram a potência do sinal recebido (RSRP em dBm). Una os logs com extend() e calcule:

- Taxa de cobertura adequada (leituras ≥ -100 dBm)
- Índice e valor do pior sinal registrado
- Média de todas as leituras

Dados de entrada

```

antena_norte (dBm): -85, -90, -105, -78, -92, -110, -88
antena_sul   (dBm): -95, -88, -102, -79, -85, -91, -97
antena_este  (dBm): -80, -75, -108, -93, -86, -103, -82
limiar de cobertura: -100 dBm

```

Saída esperada:

```

» Total de amostras      : 21
  Em cobertura (≥-100): 16
  Taxa de cobertura      : 76.2%
  Pior sinal              : -110 dBm (índice 5)
  Média                  : -91.4 dBm

```

Código Python:

```

n = [-85,-90,-105,-78,-92,-110,-88]
s = [-95,-88,-102,-79,-85,-91,-97]
l = [-80,-75,-108,-93,-86,-103,-82]

```

```

dados = n.copy()
dados.extend(s)
dados.extend(l)

cobertura = sum(1 for x in dados if x >= -100)

print("Total:", len(dados))
print("Cobertura:", cobertura)
print("Pior:", min(dados))
print("Média:", sum(dados)/len(dados))

```

Bloco 5 — Exercícios integrados `append` · `remove` · `copy` · `extend`

Ex.
13

append + remove

Eng. Mecânica — Controle de qualidade de peças usinadas

intermediário

Um lote de 10 peças é inspecionado. Use `append()` para construir separadamente as listas de aprovadas e reprovadas a partir das medições fornecidas, com base na tolerância indicada.

Dados de entrada

```

medições (mm): 50.02, 49.97, 50.06, 49.94, 50.00,
                50.08, 49.99, 50.01, 49.92, 50.04
diâmetro nominal : 50.00 mm
tolerância       : ±0.05 mm

```

Saída esperada:

```

» Aprovadas (7): [50.02, 49.97, 50.0, 49.99, 50.01, 50.04, ...]
  Reprovadas (3): [50.06, 50.08, 49.92]
  Índice de aprovação: 70%

```



Dica: Use `abs(medida - nominal) <= tolerancia` para verificar se a peça está dentro da faixa.

Código Python:

```
med = [50.02,49.97,50.06,49.94,50.00,50.08,49.99,50.01,49.92,50.04]
```

```
ok = []
```

```
nok = []
```

```

for m in med:
    if abs(m - 50.0) <= 0.05:
        ok.append(m)
    else:
        nok.append(m)

print("Aprovadas:", ok)
print("Reprovadas:", nok)

```

Ex. 14	copy + extend	Eng. Elétrica — Comparação de cenários de carga em subestação	avanzado
-----------	---------------	---	----------

Um engenheiro analisa dois cenários de carga em uma subestação: o atual e um cenário futuro com novos equipamentos. Use `copy()` para preservar o cenário atual e `extend()` para compor o futuro. Exiba um resumo comparativo com total, média e maior carga de cada cenário.

Dados de entrada
cenário atual (kW): 15.0, 22.5, 8.0, 40.0, 12.5, 30.0 novas cargas (kW): 55.0, 18.0, 75.0

Saída esperada:

»	<pre> --- Cenário atual --- Total : 128.0 kW Média : 21.3 kW Maior : 40.0 kW --- Cenário futuro --- Total : 276.0 kW Média : 30.7 kW Maior : 75.0 kW </pre>
---	---

Código Python:

```

atual = [15.0,22.5,8.0,40.0,12.5,30.0]
futuro = atual.copy()

futuro.extend([55.0,18.0,75.0])

print("Atual:", sum(atual), max(atual))
print("Futuro:", sum(futuro), max(futuro))

```

Um sistema de processamento de sinal recebe amostras de dois canais (I e Q). Escreva um programa que:

- Una os dois canais com `extend()`
- Guarde um backup com `copy()` antes de filtrar
- Calcule média (μ) e desvio padrão (σ) do sinal unido
- Remova com `remove()` todos os valores fora de $\mu \pm 3\sigma$ (ruído)
- Exiba sinal bruto, ruídos removidos e sinal limpo

Dados de entrada

```
canal_I: 0.82, 0.79, 0.85, 5.20, 0.81, 0.78, 0.83 ← 5.20 é ruído
canal_Q: 0.41, 0.39, 0.43, 0.40, -4.80, 0.42, 0.38 ← -4.80 é ruído
 $\sigma = \sqrt{(\sum (x_i - \mu)^2) / N}$  (desvio padrão amostral simples)
```

Saída esperada:

```
>> Sinal bruto (14 amostras): [0.82, 0.79, 0.85, 5.2, 0.81, ..., -4.8,
0.42, 0.38]
Ruídos removidos : [5.2, -4.8]
Sinal limpo (12 amostras): [0.82, 0.79, 0.85, 0.81, 0.78, 0.83, 0.41,
...]
```



Dica: Monte uma lista auxiliar com os ruídos antes de removê-los — nunca remova dentro do mesmo `for` que percorre a lista.

Código Python:

```
import math
```

```
I = [0.82,0.79,0.85,5.20,0.81,0.78,0.83]
```

```
Q = [0.41,0.39,0.43,0.40,-4.80,0.42,0.38]
```

```
dados = I.copy()
```

```
dados.extend(Q)
```

```
media = sum(dados)/len(dados)
```

```
variancia = sum((x-media)**2 for x in dados)/len(dados)
```

```
desvio = math.sqrt(variancia)
```

```
ruido = []

for x in dados:
    if x < media - 3*desvio or x > media + 3*desvio:
        ruido.append(x)

for r in ruido:
    dados.remove(r)

print("Ruídos:", ruido)
print("Limpo:", dados)
```